



SNS COLLEGE OF TECHNOLOGY

(An Autonomous Institution)



COIMBATORE – 35

**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND
MACHINE LEARNING**

23AMP304-WEB TECHNOLOGY LABORATORY

PRACTICAL RECORD

NAME : _____

REGISTER No : _____

CLASS : _____

SEMESTER : _____



SNS COLLEGE OF TECHNOLOGY
(An Autonomous Institution)



COIMBATORE – 35

**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND
MACHINE LEARNING**

23AMP304-WEB TECHNOLOGY LABORATORY

Name :

Class :

Branch:

Roll No :

Register No.

Certified that this is the bonafide record of work done by the above student in the
23AMP304-WEB TECHNOLOGY during the year 2025 - 2026.

Signature of Lab -in –charge

Head of the Department

Submitted for the practical examination held on

Internal Examiner

External Examiner

INDEX

[illegible]

EX.NO :01 DATE :	DESIGN MULTI-PAGE PORTFOLIO USING HTML, CSS, JAVASCRIPT
-----------------------------------	--

AIM

To design and develop a multi page personal portfolio website using HTML5 attribute semantic tags and css3 Implement a fully validated form using HTML5 attribute, javascript Validation. Apply css box model flex box / grid for the layout and ensure responsive it.

ALGORITHM

1. Create a project folder and add required files:
 - index.html
 - style.css
 - script.js
2. Design the basic structure of the webpage using HTML5 semantic tags such as:
<html>, <head>, <body>, <header>, etc.
3. Add personal details including:
 - Name
 - Contact Information
 - GitHub & LinkedIn
 - Internships
 - Skills
 - Education
 - Certifications
4. Insert a profile image.
5. Link the external CSS file using the <link> tag.

6. Design the layout using CSS properties (Box Model, Flexbox/Grid).
7. Apply hover effects and transitions to enhance UI.
8. Link the external JavaScript file.
9. Test the portfolio in the browser:
 - Verify responsiveness
 - Fix alignment issues
 - Finalize design

PROGRAM

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Portfolio - AIML Student</title>
```

```
<style>
```

```
  @import
```

```
url('https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;600&display=swap');
```

```
  * { box-sizing: border-box; }
```

```
  body {
```

```
    font-family: 'Outfit', sans-serif;
```

```
    background: linear-gradient(135deg, #09090b, #18181b);
```

```
    color: #f4f4f5;
```

```
margin: 0;
padding: 2rem 1rem;
}
```

```
.container {
  max-width: 850px;
  margin: 0 auto;
  display: flex;
  flex-direction: column;
  gap: 2.5rem;
}
```

```
.card {
  background: rgba(255, 255, 255, 0.03);
  backdrop-filter: blur(12px);
  border: 1px solid rgba(255, 255, 255, 0.08);
  border-radius: 16px;
  padding: 3rem 2.5rem;
  transition: transform 0.3s;
  box-shadow: 0 10px 30px rgba(0,0,0,0.5);
}
```

```
.card:hover {
  transform: translateY(-4px);
  border-color: rgba(99, 102, 241, 0.4);
}
```

```
.text-gradient {  
  background: linear-gradient(to right, #818cf8, #c084fc);  
  -webkit-background-clip: text;  
  -webkit-text-fill-color: transparent;  
}
```

```
h1 {  
  font-size: 2.8rem;  
  margin: 0;  
}
```

```
h2 {  
  border-bottom: 1px solid rgba(255, 255, 255, 0.1);  
  padding-bottom: 1rem;  
  font-size: 2rem;  
  display: flex;  
  justify-content: space-between;  
}
```

```
.profile {  
  text-align: center;  
  padding: 4rem 2rem;  
}
```

```
.avatar-wrapper {
```

```
display: inline-block;
position: relative;
width: 80px;
height: 80px;
margin-bottom: 1.5rem;
cursor: pointer;
}
```

```
.avatar {
  width: 100%;
  height: 100%;
  border-radius: 50%;
  border: 4px solid #818cf8;
}
```

```
.avatar-hint {
  position: absolute;
  inset: 0;
  display: flex;
  align-items: center;
  justify-content: center;
  opacity: 0;
  font-size: 0.7rem;
  background: rgba(0,0,0,0.5);
  border-radius: 50%;
  color: #fff;
}
```

```
}
```

```
.avatar-wrapper:hover .avatar-hint {  
  opacity: 1;  
}
```

```
input[type="file"] {  
  display: none;  
}
```

```
.badge {  
  display: inline-block;  
  background: rgba(99, 102, 241, 0.1);  
  color: #818cf8;  
  padding: 0.8rem 1.4rem;  
  border-radius: 99px;  
  margin: 0.4rem;  
}
```

```
.links a {  
  text-decoration: none;  
  color: #d4d4d8;  
  background: rgba(255,255,255,0.05);  
  padding: 1rem;  
  border-radius: 8px;  
}
```

```
.btn {
  background: #818cf8;
  border: none;
  padding: 0.6rem 1.4rem;
  cursor: pointer;
}

.btn:hover {
  background: #6366f1;
}

ul {
  color: #a1a1aa;
}

</style>
</head>

<body>

<div class="container">

  <div class="card profile">
    <label class="avatar-wrapper">
      

<div class="avatar-hint">Edit</div>

<input type="file" accept="image/\*" onchange="updatePfp(event)">  
</label>

<h1 class="text-gradient">Lakshana</h1>

<p>AIML 3rd Year</p>

<p>SNS College of Technology</p>

</div>

<div class="card">

<h2 class="text-gradient">About Me</h2>

<p>Passionate professional skilled in cybersecurity, data analytics, and machine learning.</p>

</div>

<div class="card">

<h2 class="text-gradient">Education</h2>

<p>HSC (98%)</p>

<p>SSLC (99%)</p>

</div>

<div class="card">

<h2 class="text-gradient">Skills</h2>

<span class="badge">Machine Learning</span>

```
Data Structure
</div>
```

```
<div class="card">
 <h2 class="text-gradient">Contact</h2>
 <p>Phone: +91 XXXXXXXXX</p>
 <p>Email: example@gmail.com</p>
</div>
```

```
<div class="card">
 <h2 class="text-gradient">
 Extra Details
 <button class="btn" onclick="addDetail()">+ Add</button>
 </h2>
```

```
<ul id="dynamic-list">
 Participated in Hackathon

</div>
```

```
</div>
```

```
<script>
function updatePfp(event) {
 const file = event.target.files[0];
 if (file) {
```

```
const reader = new FileReader();
reader.onload = function(e) {
 document.getElementById('pfp').src = e.target.result;
};
reader.readAsDataURL(file);
}
}

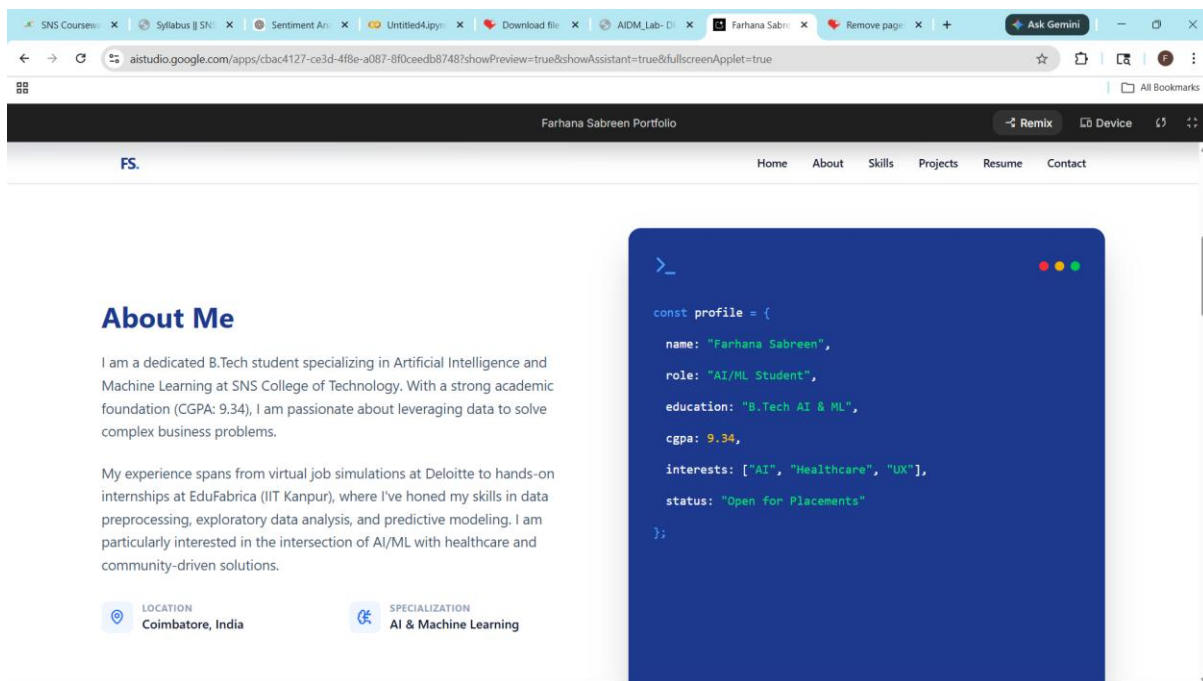
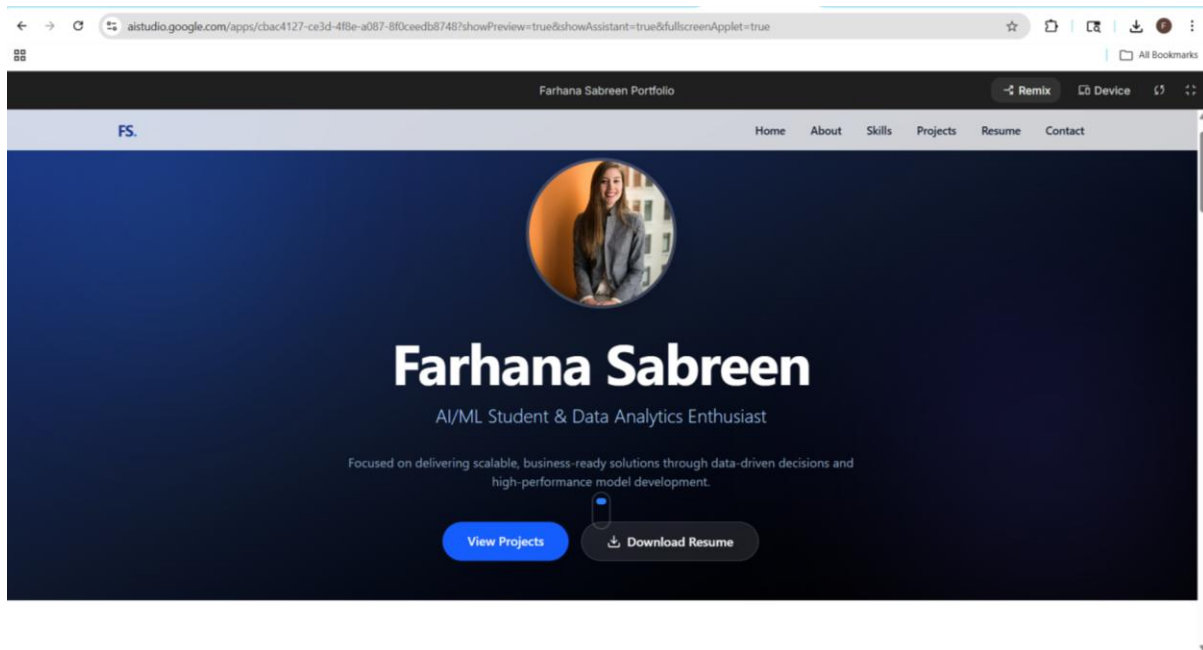
function addDetail() {
 const val = prompt("Enter detail:");
 if (val) {
 const li = document.createElement('li');
 li.innerText = val;
 document.getElementById('dynamic-list').appendChild(li);
 }
}

</script>

</body>

</html>
```

## OUTPUT



## RESULT

Thus, the multi-page personal portfolio website was designed and developed successfully using HTML5, CSS3, and JavaScript, and the output was verified.

<b>EX.NO :02</b> <b>DATE :</b>	<b>CREATE A DYNAMIC TASK MANAGEMENT SYSTEM USING JAVASCRIPT</b>
-----------------------------------	---------------------------------------------------------------------

## **AIM**

To create a dynamic Task Management Application using JavaScript by implementing features such as adding tasks, marking completion, and deleting tasks through event-driven DOM manipulation. To use JavaScript arrays for in-memory data management and browser local storage for data persistence.

## **ALGORITHM**

1. Create the HTML file and design the basic structure including:
  - Input box
  - “Add Task” button
  - Empty list to display tasks
2. Create a CSS file to apply styling such as:
  - Layout alignment
  - Button design
  - Task card design
  - Completed task highlighting
3. Create a JavaScript file and declare an empty array tasks to store task data in memory.
4. Access HTML elements:
  - Task input field
  - Add button
  - Task list container
5. Load previously saved tasks from local storage.

6. Create a function to display tasks dynamically.
7. Implement:
  - Task completion functionality
  - Task deletion functionality
8. Save and refresh updated tasks in local storage.

## PROGRAM

### index.html

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Task Management App</title>

<link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">

 <h1>Task Management Application</h1>

 <div class="task-input">

 <input type="text" id="taskInput" placeholder="Enter a new task...">

 <button id="addTaskBtn">Add Task</button>

 </div>
```

```
 <ul id="taskList">
</div>

<script src="script.js"></script>
</body>
</html>
```

### **style.css**

```
* {
margin: 0;
padding: 0;
box-sizing: border-box;
font-family: Arial, sans-serif;
}
```

```
body {
background: #f4f6f9;
display: flex;
justify-content: center;
align-items: center;
height: 100vh;
}
```

```
.container {
background: white;
width: 450px;
padding: 25px;
```

```
border-radius: 10px;
box-shadow: 0px 4px 10px rgba(0,0,0,0.2);
}
```

```
h1 {
 text-align: center;
 margin-bottom: 20px;
 color: #333;
}
```

```
.task-input {
 display: flex;
 gap: 10px;
 margin-bottom: 20px;
}
```

```
.task-input input {
 flex: 1;
 padding: 10px;
 border-radius: 6px;
 border: 1px solid #ccc;
 outline: none;
}
```

```
.task-input button {
 padding: 10px 15px;
```

```
border: none;
background: #007bff;
color: white;
border-radius: 6px;
cursor: pointer;
font-weight: bold;
}
```

```
.task-input button:hover {
background: #0056b3;
}
```

```
#taskList {
list-style: none;
}
```

```
.task-item {
display: flex;
justify-content: space-between;
align-items: center;
background: #f9f9f9;
padding: 12px;
border-radius: 6px;
margin-bottom: 10px;
border-left: 5px solid #007bff;
}
```

```
.task-item.completed {
 text-decoration: line-through;
 opacity: 0.7;
 border-left: 5px solid green;
}
```

```
.task-actions button {
 border: none;
 padding: 6px 10px;
 margin-left: 5px;
 border-radius: 5px;
 cursor: pointer;
 font-size: 14px;
}
```

```
.complete-btn {
 background: green;
 color: white;
}
```

```
.delete-btn {
 background: red;
 color: white;
}
```

```
.complete-btn:hover {
 background: darkgreen;
}
```

```
.delete-btn:hover {
 background: darkred;
}
```

### **Script.js**

```
let tasks = [];
```

```
const taskInput = document.getElementById("taskInput");
const addTaskBtn = document.getElementById("addTaskBtn");
const taskList = document.getElementById("taskList");
```

```
function loadTasks() {
 const storedTasks = localStorage.getItem("tasks");
 if (storedTasks) {
 tasks = JSON.parse(storedTasks);
 }
}
```

```
function saveTasks() {
 localStorage.setItem("tasks", JSON.stringify(tasks));
}
```

```
function displayTasks() {
```

```
taskList.innerHTML = "";
```

```
tasks.forEach((task, index) => {
```

```
 const li = document.createElement("li");
```

```
 li.className = "task-item";
```

```
 if (task.completed) {
```

```
 li.classList.add("completed");
```

```
 }
```

```
 li.innerHTML = `
```

```
 ${task.name}
```

```
 <div class="task-actions">
```

```
 <button class="complete-btn">Complete</button>
```

```
 <button class="delete-btn">Delete</button>
```

```
 </div>
```

```
`;
```

```
 li.querySelector(".complete-btn")
```

```
 .addEventListener("click", () => toggleCompleteTask(index));
```

```
 li.querySelector(".delete-btn")
```

```
 .addEventListener("click", () => deleteTask(index));
```

```
 taskList.appendChild(li);
```

```
});
```

```
}
```

```
function addTask() {
 const taskName = taskInput.value.trim();
```

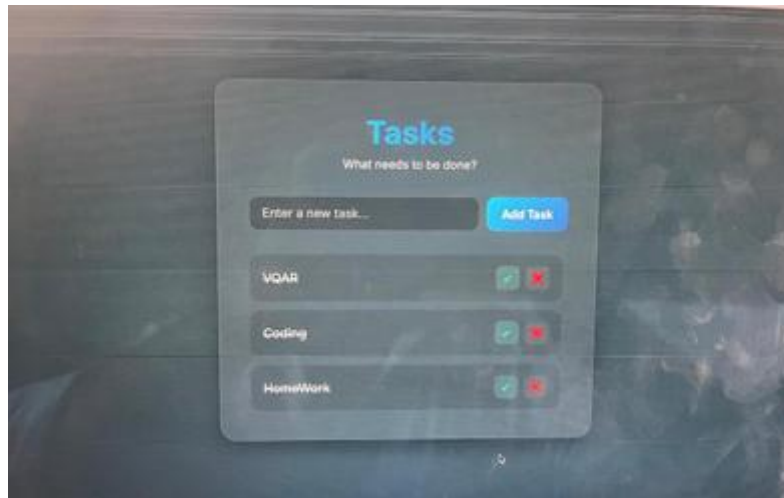
```
 if (taskName === "") {
 alert("Please enter a valid task!");
 return;
 }
```

```
 const newTask = {
 name: taskName,
 completed: false
 };
```

```
 tasks.push(newTask);
 saveTasks();
 displayTasks();
 taskInput.value = "";
}
```

```
function toggleCompleteTask(index) {
 tasks[index].completed = !tasks[index].completed;
 saveTasks();
 displayTasks();
}
```

## OUTPUT



## RESULT

Thus, the dynamic Task Management Application was successfully created using HTML, CSS, and JavaScript. Tasks can be added, marked as completed, deleted, and stored using local storage, and the output is verified.

<b>EX.NO :03</b> <b>DATE :</b>	<b>DEVELOPING A REACT JS APPLICATION TO DEMONSTRATE FUNDAMENTAL CONCEPTS OF REACT</b>
-----------------------------------	-----------------------------------------------------------------------------------------------

## AIM

To develop a React JS application to demonstrate fundamental concepts such as components, state management, props, event handling, and form handling.

## ALGORITHM

### Procedure:

1. Install **Node.js** and **npm** to set up the React environment.
2. Create a new React application using:  

```
npx create-react-app react-demo
```
3. Open the project folder in Visual Studio Code.
4. Run the application using:  

```
npm start
```
5. Create components such as **Counter**, **UserForm**, and **DisplayUser**.
6. Use **JSX** in App.js to structure the UI.
7. Implement a **Counter component** using the useState hook.
8. Create a **UserForm component** to accept name and age.
9. Pass user data using **props** to the DisplayUser component.
10. Verify that counter, form input, and display functionalities work correctly.

## PROGRAM

### App.js

```
import React, { useState } from "react";
import Counter from "./Counter";
import UserForm from "./UserForm";
```

```
import DisplayUser from "../DisplayUser";
```

```
function App() {
```

```
 const [user, setUser] = useState({ name: "", age: "" });
```

```
 return (
```

```
 <div style={{
```

```
 textAlign: "center",
```

```
 fontFamily: "Arial",
```

```
 backgroundColor: "#f5f5f5",
```

```
 minHeight: "100vh",
```

```
 padding: "20px"
```

```
 }}>
```

```
 <h1 style={{ color: "#333", marginBottom: "30px" }}>
```

```
 Welcome to React Application
```

```
 </h1>
```

```
 <div style={{ backgroundColor: "white", padding: "20px", marginBottom: "20px" }}>
```

```
 <Counter />
```

```
 </div>
```

```
 <div style={{ backgroundColor: "white", padding: "20px", marginBottom: "20px" }}>
```

```
 <UserForm setUser={setUser} />
```

```
 </div>
```

```
 <div style={{ backgroundColor: "white", padding: "20px" }}>
 <DisplayUser user={user} />
 </div>
 </div>
);
}
```

```
export default App;
```

### **Counter.js**

```
import React, { useState } from "react";

function Counter() {
 const [count, setCount] = useState(0);

 return (
 <div>
 <h2>Counter Example</h2>
 <h3>{count}</h3>
 <button onClick={() => setCount(count + 1)}>
 Increase Counter
 </button>
 </div>
);
}

export default Counter;
```

## UserForm.js

```
import React, { useState } from "react";

function UserForm({ setUser }) {
 const [name, setName] = useState("");
 const [age, setAge] = useState("");

 function handleSubmit(e) {
 e.preventDefault();
 setUser({ name, age });
 }

 return (
 <div>
 <h2>Enter User Details</h2>
 <form onSubmit={handleSubmit}>
 <input
 type="text"
 placeholder="Enter Name"
 value={name}
 onChange={(e) => setName(e.target.value)}
 required
 />


```

```
<input
 type="number"
 placeholder="Enter Age"
 value={age}
 onChange={(e) => setAge(e.target.value)}
 required
/>

<button type="submit">Submit</button>
</form>
</div>
);
}
```

```
export default UserForm;
```

### **index.js**

```
import React from "react";
```

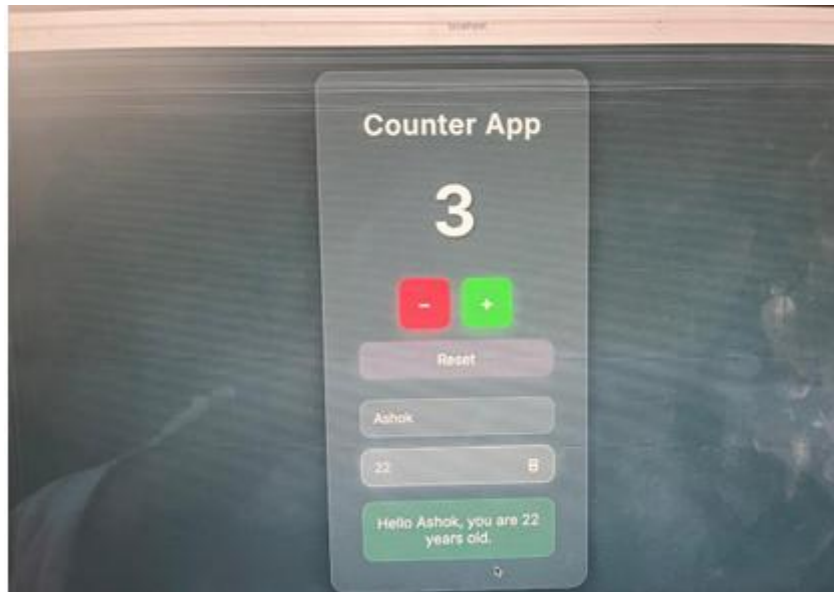
```
import ReactDOM from "react-dom/client";
```

```
import App from "./App";
```

```
const root = ReactDOM.createRoot(document.getElementById("root"));
```

```
root.render(<App />);
```

## OUTPUT



## RESULT

Thus, the React JS application was successfully developed and executed. The fundamental concepts such as components, state, props, and event handling were implemented and verified.

<b>EX.NO :04</b> <b>DATE :</b>	<b>SETUP A NODE.JS PROJECT WITH EXPRESS.JS FRAMEWORK</b>
-----------------------------------	----------------------------------------------------------

## AIM

To design and implement a RESTful API using Node.js and Express.js with GET, POST, PUT, and DELETE methods for resource management. To test all endpoints using API client tools like Postman or Thunder Client and integrate a lightweight SQLite database.

## ALGORITHM

### Procedure:

1. Install **Node.js** and required packages.
2. Initialize the project using:  

```
npm init -y
```
3. Install dependencies:  

```
npm install express sqlite3 body-parser cors
```
4. Create a server using **Express.js**.
5. Connect to the **SQLite** database.
6. Create a table to store student data.
7. Implement **GET** method to retrieve data.
8. Implement **POST, PUT, DELETE** methods for CRUD operations.
9. Test API endpoints using Postman or Thunder Client.
10. Run the server and verify outputs.

## PROGRAM

### server.js

```
const express = require("express");
const sqlite3 = require("sqlite3").verbose();
```

```
const bodyParser = require("body-parser");
const cors = require("cors");

const app = express();

// Middleware
app.use(bodyParser.json());
app.use(cors());

// Default route
app.get("/", (req, res) => {
 res.send("Welcome to the Student API! Use /students endpoint.");
});

// Database connection
const db = new sqlite3.Database("students.db");

// Create table
db.run(`
 CREATE TABLE IF NOT EXISTS students (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 name TEXT,
 age INTEGER,
 course TEXT
)
`);
```

```
// GET all students

app.get("/students", (req, res) => {
 db.all("SELECT * FROM students", [], (err, rows) => {
 if (err) return res.send(err);
 res.json(rows);
 });
});

// GET student by ID

app.get("/students/:id", (req, res) => {
 db.get("SELECT * FROM students WHERE id = ?", [req.params.id], (err,
row) => {
 if (err) return res.send(err);
 res.json(row);
 });
});

// POST - Add new student

app.post("/students", (req, res) => {
 const { name, age, course } = req.body;

 db.run(
 "INSERT INTO students (name, age, course) VALUES (?, ?, ?)",
 [name, age, course],
 function (err) {
 if (err) return res.send(err);
 }
);
});
```

```
 res.json({ id: this.lastID });
 }
);
});
```

// PUT - Update student

```
app.put("/students/:id", (req, res) => {
 const { name, age, course } = req.body;

 db.run(
 "UPDATE students SET name=?, age=?, course=? WHERE id=?",
 [name, age, course, req.params.id],
 function (err) {
 if (err) return res.send(err);
 res.json({ message: "Student updated successfully" });
 }
);
});
```

// DELETE - Delete student

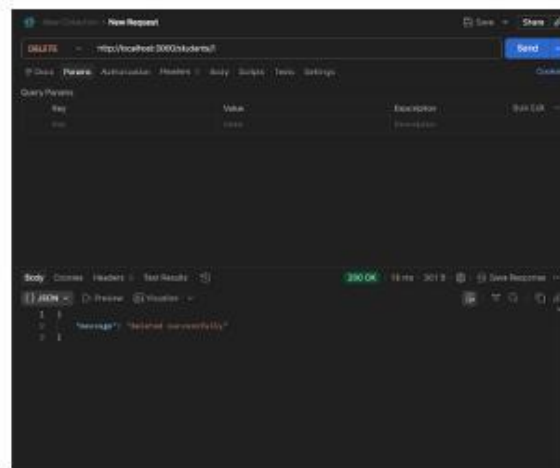
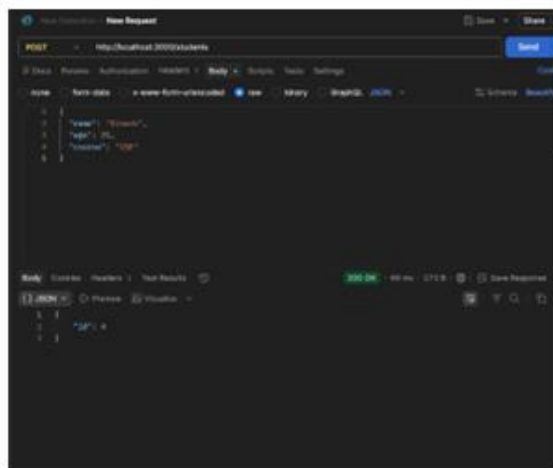
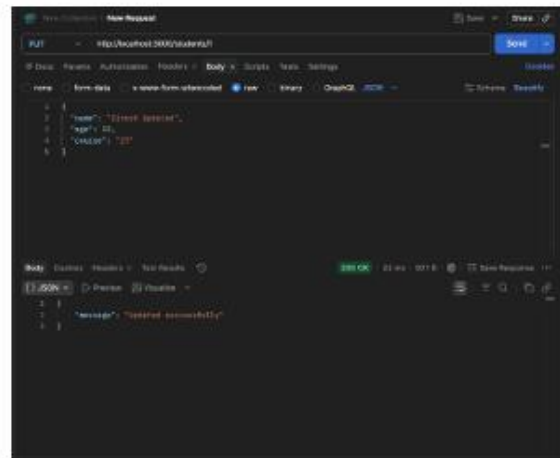
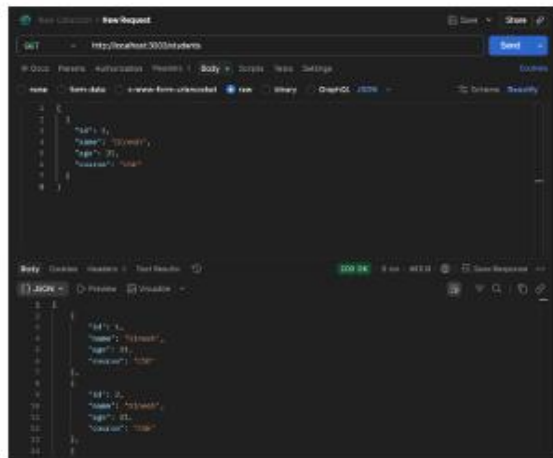
```
app.delete("/students/:id", (req, res) => {
 db.run("DELETE FROM students WHERE id=?", [req.params.id], function
(err) {
 if (err) return res.send(err);
 res.json({ message: "Student deleted successfully" });
 });
});
```

```
// Start server
app.listen(3000, () => {
 console.log("Server running on http://localhost:3000");
});
```

### **package.json**

```
{
 "name": "wt-exp4",
 "version": "1.0.0",
 "description": "Express SQLite CRUD API",
 "main": "server.js",
 "scripts": {
 "start": "node server.js"
 },
 "keywords": [],
 "author": "",
 "license": "ISC",
 "type": "commonjs",
 "dependencies": {
 "body-parser": "^1.20.2",
 "cors": "^2.8.5",
 "express": "^4.18.2",
 "sqlite3": "^5.1.6"
 }
}
```

## OUTPUT



## RESULT

Thus, the Node.js project using Express.js framework was successfully implemented. CRUD operations were performed using RESTful APIs, and the results were verified using API testing tools.

<b>EX.NO :05</b> <b>DATE :</b>	<b>INTEGRATION OF PRE-TRAINED          TENSORFLOW.JS MODEL (MOBILENET) FOR          IMAGE CLASSIFICATION</b>
-----------------------------------	----------------------------------------------------------------------------------------------------------------------

## AIM

To build a web interface that allows users to upload an image and dynamically display model predictions along with confidence scores using a pre-trained TensorFlow.js MobileNet model

## ALGORITHM

1. Start the process.
2. Include TensorFlow.js library in the web page.
3. Load a pre-trained model such as MobileNet.
4. Design an HTML interface for image upload.
5. Capture user input image.
6. Preprocess the image according to model requirements.
7. Pass the image to the loaded model for prediction.
8. Retrieve prediction results with confidence scores.
9. Display results dynamically on the web page.
10. Stop the process.

## PROGRAM

### Index.html

```

<!DOCTYPE html>

<html lang="en">

<head>

 <meta charset="UTF-8">

 <meta name="viewport" content="width=device-width, initial-scale=1.0">

```

<title>Image Classification - MobileNet</title>

<!-- TensorFlow.js -->

<script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs@latest"></script>

<!-- MobileNet Model -->

<script src="https://cdn.jsdelivr.net/npm/@tensorflow-models/mobilenet"></script>

<link rel="stylesheet" href="style.css">

</head>

<body>

<h2>Image Classification using MobileNet</h2>

<input type="file" id="imageUpload" accept="image/\*">

<br><br>

<img id="preview" width="300" />

<h3>Predictions:</h3>

<div id="results"></div>

<script src="script.js"></script>

</body>

</html>

## **style.css**

```
body {
 font-family: Arial, sans-serif;
 text-align: center;
 margin-top: 40px;
}
```

```
#results {
 margin-top: 20px;
 font-size: 18px;
}
```

## **script.js**

```
let model;
```

```
// Load MobileNet model
```

```
async function loadModel() {
 model = await mobilenet.load();
 console.log("MobileNet Model Loaded");
}
```

```
loadModel();
```

```
// Handle image upload
```

```
document.getElementById("imageUpload").addEventListener("change", async
function (event) {
```

```
 const file = event.target.files[0];
```

```
const reader = new FileReader();

reader.onload = async function () {
 const img = document.getElementById("preview");
 img.src = reader.result;

 img.onload = async function () {
 const predictions = await model.classify(img);

 const resultsDiv = document.getElementById("results");
 resultsDiv.innerHTML = "";

 predictions.forEach(pred => {
 resultsDiv.innerHTML += `
 <p>${pred.className} - Confidence: ${
(pred.probability
* 100).toFixed(2)}%</p>
 `;
 });
 };
};

reader.readAsDataURL(file);
});
```

## OUTPUT



## RESULT

Thus, the TensorFlow.js MobileNet model was successfully integrated into a web application. The system correctly classified uploaded images and displayed predictions along with confidence scores dynamically.

<b>EX.NO :06</b> <b>DATE :</b>	<b>REPLICATING A PORTFOLIO WEBSITE USING A DRAG-AND-DROP BUILDER APPROACH</b>
-----------------------------------	-----------------------------------------------------------------------------------

## AIM

To build a portfolio website incorporating features like contact forms, SEO settings, and widgets using a visual (drag-and-drop) approach, and to analyze the trade-offs between hand-coding and visual website builders.

## ALGORITHM

1. Start the process.
2. Open a website builder platform (e.g., Wix or similar) and log in.
3. Choose a suitable portfolio template.
4. Use drag-and-drop tools to design the webpage layout.
5. Add content such as text, images, and project details.
6. Integrate a contact form for user interaction.
7. Apply SEO settings (meta tags, keywords, description).
8. Add widgets such as social links, galleries, and animations.
9. Publish the website and compare it with a hand-coded version.
10. Stop the process.

## PROGRAM

```

<!DOCTYPE html>

<html lang="en">

<head>

 <meta charset="UTF-8">

 <!-- SEO Meta Tags -->

```

```
<meta name="description" content="Personal portfolio showcasing web
development, AI projects and Cyber Security.">

<meta name="keywords" content="portfolio, web developer, AI, projects">

<meta name="author" content="Lakshana">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Lakshana Portfolio | Web Developer</title>

<link rel="stylesheet" href="style.css">

</head>

<body>

<header>

 <h1>Lakshana</h1>

 <nav>

 Home

 About

 Skills

 Projects

 Contact

 </nav>

</header>

<section id="home" class="section">

 <h2>Welcome to My Portfolio</h2>

 <p>Web Developer | AI Enthusiast | Problem Solver</p>
```

</section>

<section id="about" class="section">

<h2>About Me</h2>

<p>I am a passionate developer skilled in web technologies, Cyber Security and AI-based systems.</p>

</section>

<section id="skills" class="section">

<h2>Skills</h2>

<ul>

<li>HTML, CSS, JavaScript</li>

<li>Python</li>

<li>Machine Learning</li>

<li>Cyber Security</li>

</ul>

</section>

<section id="projects" class="section">

<h2>Projects</h2>

<div class="project">

<h3>AI SPACEMARK</h3>

<p>File Organising System</p>

</div>

<div class="project">

<h3>Voice Recognition</h3>

<p>Implemented MobileNet using TensorFlow.js</p>

</div>

</section>

<section id="contact" class="section">

<h2>Contact Me</h2>

<form id="contactForm">

<input type="text" id="name" placeholder="Your Name" required>

<input type="email" id="email" placeholder="Your Email" required>

<textarea id="message" placeholder="Your Message" required></textarea>

<button type="submit">Send Message</button>

</form>

<p id="formMessage"></p>

</section>

<footer>

<p>© 2026 Lakshana | All Rights Reserved</p>

</footer>

<script src="script.js"></script>

</body>

```
</html>
```

```
body {
 font-family: Arial, sans-serif;
 margin: 0;
 padding: 0;
 scroll-behavior: smooth;
}
```

```
header {
 background: #222;
 color: white;
 padding: 15px;
 text-align: center;
}
```

```
nav a {
 color: white;
 margin: 0 15px;
 text-decoration: none;
}
```

```
.section {
 padding: 50px;
 text-align: center;
}
```

```
#skills ul {
 list-style: none;
 padding: 0;
}
```

```
.project {
 background: #eaeaea;
 margin: 15px;
 padding: 15px;
 border-radius: 5px;
}
```

```
form {
 max-width: 400px;
 margin: auto;
}
```

```
input, textarea {
 width: 100%;
 padding: 10px;
 margin: 10px 0;
}
```

```
button {
 padding: 10px 20px;
 background: #333;
```

```
 color: white;
 border: none;
 cursor: pointer;
}
```

```
button:hover {
 background: #555;
}
```

```
footer {
 background: #222;
 color: white;
 padding: 10px;
 text-align: center;
}
```

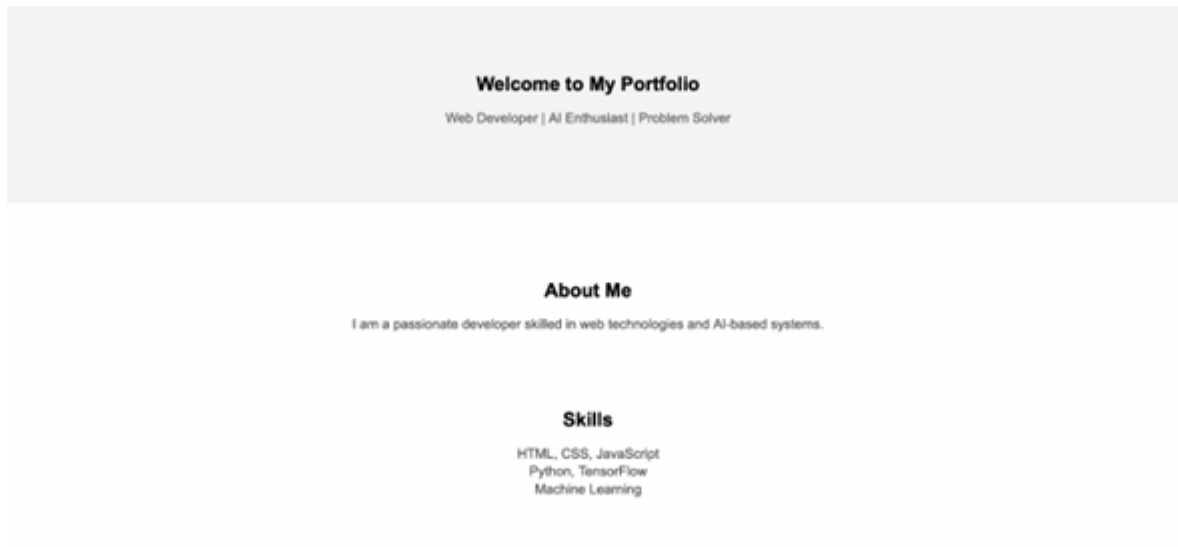
```
document.getElementById("contactForm").addEventListener("submit",
function (event) {
 event.preventDefault();

 const name = document.getElementById("name").value;

 document.getElementById("formMessage").innerHTML =
 "Thank you, " + name + "! Your message has been received.";

 this.reset();
});
```

## OUTPUT



## RESULT

Thus, the portfolio website was successfully developed using a drag-and-drop builder approach and replicated using HTML, CSS, and JavaScript. The features such as contact form, SEO meta tags, and responsive layout were implemented and verified.

**EX.NO :07**

**DATE :**

**DESIGN AND CODE A FUNCTIONAL CHAT  
INTERFACE PROTOTYPE**

**AIM**

To develop a chatbot interface using JavaScript with predefined responses and keyword matching to simulate real-time AI interaction using frontend logic.

**ALGORITHM**

1. Start the process.
2. Design the chat interface using HTML and CSS.
3. Create an input box and send button for user messages.
4. Use JavaScript to capture user input dynamically.
5. Define predefined responses for chatbot interaction.
6. Implement keyword matching logic in JavaScript.
7. Process user messages and identify matching responses.
8. Display user messages and bot replies in the chat window.
9. Update the chat interface dynamically without reloading the page.
10. Stop the process.

**PROGRAM**

**index.html**

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
 <meta charset="UTF-8">
```

```
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
 <title>Chatbot Interface</title>
```

```
 <link rel="stylesheet" href="style.css">
```

```
</head>

<body>

<div class="chat-container">
 <h2>Chatbot Prototype</h2>

 <div id="chatBox" class="chat-box"></div>

 <div class="input-area">
 <input type="text" id="userInput" placeholder="Type your message...">
 <button onclick="sendMessage()">Send</button>
 </div>
</div>

<script src="script.js"></script>
</body>
</html>
```

### **style.css**

```
body {
 font-family: Arial, sans-serif;
 background: #f4f6f9;
 display: flex;
 justify-content: center;
 align-items: center;
 height: 100vh;
}
```

```
.chat-container {
 width: 400px;
 background: white;
 border-radius: 10px;
 padding: 20px;
 box-shadow: 0 4px 10px rgba(0,0,0,0.2);
}
```

```
.chat-box {
 height: 300px;
 overflow-y: auto;
 border: 1px solid #ccc;
 padding: 10px;
 margin-bottom: 10px;
}
```

```
.user {
 text-align: right;
 color: blue;
 margin: 5px 0;
}
```

```
.bot {
 text-align: left;
 color: green;
```

## **script.js**

```
function sendMessage() {
 const input = document.getElementById("userInput");
 const message = input.value.trim();

 if (message === "") return;

 displayMessage(message, "user");
 input.value = "";

 const response = getBotResponse(message);

 setTimeout(() => {
 displayMessage(response, "bot");
 }, 500);
}

function displayMessage(message, sender) {
 const chatBox = document.getElementById("chatBox");

 const msgDiv = document.createElement("div");
 msgDiv.className = sender;
 msgDiv.innerText = message;

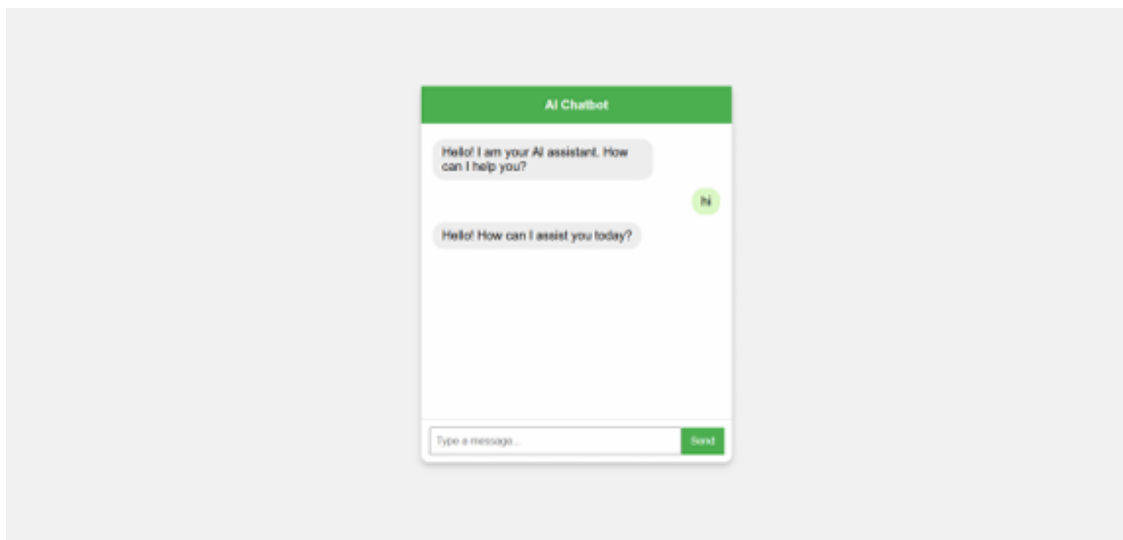
 chatBox.appendChild(msgDiv);
 chatBox.scrollTop = chatBox.scrollHeight;
```

```
}
```

```
function getBotResponse(message) {
 message = message.toLowerCase();

 if (message.includes("hello") || message.includes("hi")) {
 return "Hello! How can I help you?";
 }
 else if (message.includes("name")) {
 return "I am a simple chatbot prototype.";
 }
}
```

## OUTPUT



## RESULT

Thus, the chatbot interface prototype was successfully designed and implemented. The system was able to process user input, perform keyword-based matching, and generate appropriate responses dynamically without page reload.

<b>EX.NO :08</b> <b>DATE :</b>	<b>IMPLEMENTATION OF SIMULATED CONTENT-AWARE FILL USING IMAGE PROCESSING</b>
-----------------------------------	--------------------------------------------------------------------------------------

## AIM

To implement a simulated version of Adobe Sensei signature Content-Aware Fill functionality using image processing techniques and analyze its working using client-side JavaScript.

## ALGORITHM

1. Start the process.
2. Load an input image into the web application.
3. Allow the user to select/remove an unwanted region.
4. Use JavaScript and HTML5 Canvas to process the image.
5. Detect surrounding pixels of the selected region.
6. Apply interpolation or patch-based filling technique.
7. Replace the removed area using nearby pixel information.
8. Render the updated image dynamically on the screen.
9. Compare with Adobe Sensei cloud-based approach.
10. Stop the process.

## PROGRAM

### Index.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Simulated Content-Aware Fill</title>
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<link rel="stylesheet" href="style.css">
</head>
<body>

<h2>Content-Aware Fill (Simulation)</h2>

<input type="file" id="imageUpload" accept="image/*">

<canvas id="canvas"></canvas>

<button onclick="fillArea()">Apply Content-Aware Fill</button>
<button onclick="resetImage()">Reset</button>

<script src="script.js"></script>

</body>
</html>
```

### **Script.js**

```
const canvas = document.getElementById("canvas");
const ctx = canvas.getContext("2d");

let originalImage = null;
let maskArea = [];

const MAX_WIDTH = 800;
const MAX_HEIGHT = 500;
```

```
// Load Image

document.getElementById("imageUpload").addEventListener("change",
function (e) {

 const file = e.target.files[0];

 if (!file) return;

 const reader = new FileReader();

 reader.onload = function (event) {

 const img = new Image();

 img.onload = function () {

 let width = img.width;

 let height = img.height;

 const widthRatio = MAX_WIDTH / width;
 const heightRatio = MAX_HEIGHT / height;
 const scale = Math.min(widthRatio, heightRatio, 1);

 width = width * scale;
 height = height * scale;

 canvas.width = width;
 canvas.height = height;

 ctx.clearRect(0, 0, canvas.width, canvas.height);
```

```
ctx.drawImage(img, 0, 0, width, height);
```

```
originalImage = ctx.getImageData(0, 0, canvas.width, canvas.height);
```

```
maskArea = [];
```

```
};
```

```
img.src = event.target.result;
```

```
};
```

```
reader.readAsDataURL(file);
```

```
});
```

```
// Select Area
```

```
canvas.addEventListener("click", function (e) {
```

```
 const rect = canvas.getBoundingClientRect();
```

```
 const scaleX = canvas.width / rect.width;
```

```
 const scaleY = canvas.height / rect.height;
```

```
 const x = Math.floor((e.clientX - rect.left) * scaleX);
```

```
 const y = Math.floor((e.clientY - rect.top) * scaleY);
```

```
 ctx.fillStyle = "red";
```

```
 ctx.fillRect(x - 5, y - 5, 10, 10);
```

```
 maskArea.push({ x, y });
```

```
});
```

```
// Fill Area
```

```
function fillArea() {
```

```
 if (!originalImage || maskArea.length === 0) {
```

```
 alert("Please upload image and select area first.");
```

```
 return;
```

```
 }
```

```
 const imageData = ctx.getImageData(0, 0, canvas.width, canvas.height);
```

```
 const data = imageData.data;
```

```
 maskArea.forEach(point => {
```

```
 const fillRadius = 12;
```

```
 const sampleRadius = 30;
```

```
 let r = 0, g = 0, b = 0, count = 0;
```

```
 for (let dx = -sampleRadius; dx <= sampleRadius; dx++) {
```

```
 for (let dy = -sampleRadius; dy <= sampleRadius; dy++) {
```

```
 const nx = point.x + dx;
```

```
 const ny = point.y + dy;
```

```
 if (
```

```
 nx >= 0 && ny >= 0 &&
```

```
 nx < canvas.width && ny < canvas.height &&
```

```

 Math.sqrt(dx * dx + dy * dy) > fillRadius
) {
 const index = (ny * canvas.width + nx) * 4;

 r += data[index];
 g += data[index + 1];
 b += data[index + 2];
 count++;
 }
}

if (count === 0) return;

const avgR = r / count;
const avgG = g / count;
const avgB = b / count;

for (let dx = -fillRadius; dx <= fillRadius; dx++) {
 for (let dy = -fillRadius; dy <= fillRadius; dy++) {
 const fx = point.x + dx;
 const fy = point.y + dy;

 if (
 fx >= 0 && fy >= 0 &&
 fx < canvas.width && fy < canvas.height &&

```

```

 Math.sqrt(dx * dx + dy * dy) <= fillRadius
) {
 const fillIndex = (fy * canvas.width + fx) * 4;

 data[fillIndex] = avgR;
 data[fillIndex + 1] = avgG;
 data[fillIndex + 2] = avgB;
 }
}
});

ctx.putImageData(imageData, 0, 0);
maskArea = [];
}

```

// Reset Image

```

function resetImage() {
 if (originalImage) {
 ctx.putImageData(originalImage, 0, 0);
 maskArea = [];
 }
}

```

## Style.css

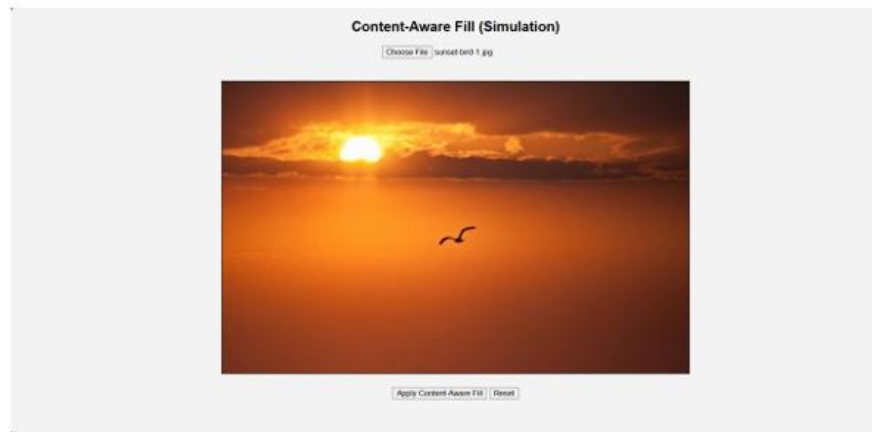
```

body {
 font-family: Arial;

```

## OUTPUT

### Before Content-Aware Fill



### After Content-Aware Fill



## RESULT

Thus, the simulated Content-Aware Fill application was implemented successfully and the output was verified.

<b>EX.NO :09</b> <b>DATE :</b>	<b>SENTIMENT ANALYSIS WEB APPLICATION USING TENSORFLOW.JS</b>
-----------------------------------	-------------------------------------------------------------------

## **AIM**

To design and develop a web application that performs real-time sentiment analysis on user input text using a TensorFlow.js model.

## **ALGORITHM**

1. Start the process.
2. Include TensorFlow.js in the web page.
3. Load a pre-trained sentiment analysis model.
4. Design a responsive UI using HTML and CSS.
5. Create an input field for user text.
6. Capture input using JavaScript.
7. Preprocess text as required by the model.
8. Pass input to the model for sentiment prediction.
9. Display the result with confidence score.
10. Stop the process.

## **PROGRAM**

### **Index.html**

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
 <meta charset="UTF-8">
```

```
<title>Sentiment Analysis Web App</title>

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<!-- TensorFlow.js -->

<script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs@4.16.0"></script>

<script src="https://cdn.jsdelivr.net/npm/@tensorflow-models/universal-
sentence-encoder"></script>

<style>
 body {
 font-family: Arial;
 background: linear-gradient(135deg, #4CAF50, #2196F3);
 display: flex;
 justify-content: center;
 align-items: center;
 height: 100vh;
 margin: 0;
 }

 .container {
 background: white;
 padding: 30px;
 width: 90%;
 max-width: 500px;
 border-radius: 10px;
 box-shadow: 0 5px 15px rgba(0,0,0,0.3);
 text-align: center;
```

```
}
```

```
textarea {
 width: 100%;
 height: 100px;
 padding: 10px;
 font-size: 16px;
}
```

```
button {
 margin-top: 15px;
 padding: 10px 20px;
 background: #4CAF50;
 border: none;
 color: white;
 cursor: pointer;
}
```

```
.result {
 margin-top: 20px;
 font-size: 18px;
 padding: 10px;
 border-radius: 8px;
}
```

```
</style>
```

```
</head>
```

```
<body>

<div class="container">

 <h2>Real-Time Sentiment Analysis</h2>

 <textarea id="textInput" placeholder="Enter your review here..."></textarea>

 <button onclick="analyze()">Analyze Sentiment</button>

 <div id="result" class="result"></div>

</div>

<script>

let model;

// Basic word-based sentiment logic (simulation)

let positiveWords = ["good", "great", "excellent", "love", "amazing",
"awesome", "happy", "best", "fantastic"];

let negativeWords = ["bad", "worst", "hate", "terrible", "poor", "awful", "sad",
"angry"];

async function loadModel() {

 document.getElementById("result").innerHTML = "Loading AI model...";

 // Load Universal Sentence Encoder (not directly used but shows TF.js
integration)

 model = await use.load();
```

```
document.getElementById("result").innerHTML = "Model Loaded. Enter
text.";
}
```

```
loadModel();
```

```
positiveWords.forEach(word => {
 if (text.includes(word)) score += 1;
});
```

```
negativeWords.forEach(word => {
 if (text.includes(word)) score -= 1;
});
```

```
let sentiment = "";
```

```
let confidence = Math.min(Math.abs(score) / 3, 1).toFixed(2);
```

```
if (score > 0) {
 sentiment = "Positive";
 resultDiv.style.background = "#C8E6C9";
} else if (score < 0) {
 sentiment = "Negative";
 resultDiv.style.background = "#FFCDD2";
} else {
 sentiment = "Neutral";
 resultDiv.style.background = "#FFF9C4";
}
```

```
resultDiv.innerHTML = `
 Sentiment: ${sentiment}

 Confidence: ${confidence}
`;
}
</script>
</body>
</html>
```

## OUTPUT



## RESULT

Thus, the Sentiment Analysis web application was implemented successfully and the output was verified.

<b>EX.NO :10</b> <b>DATE :</b>	<b>AI CHATBOT WEBPAGE USING FIREDROP CONCEPT</b>
-----------------------------------	------------------------------------------------------

## AIM

To develop a smart chatbot interface prototype inspired by FireDrop AI for assisting in website creation.

## ALGORITHM

1. Start the process.
2. Design the chat interface using HTML and CSS.
3. Create input field and send button for user interaction.
4. Use JavaScript to capture and display user messages.
5. Implement keyword-based response logic for chatbot.
6. Add basic NLP techniques for better understanding of input.
7. Simulate FireDrop-like suggestions.
8. Generate dynamic content responses.
9. Maintain conversational flow for AI-assisted website building.
10. Stop the process.

## PROGRAM

### Program:

### Index.html

```
<!DOCTYPE html>
<html>
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>FireDrop Inspired AI Chatbot</title>
```

```
<style>
```

```
body {
```

```
 margin: 0;
```

```
 font-family: Arial, sans-serif;
```

```
 background: linear-gradient(135deg, #667eea, #764ba2);
```

```
 display: flex;
```

```
 justify-content: center;
```

```
 align-items: center;
```

```
 height: 100vh;
```

```
}
```

```
.chat-container {
```

```
 width: 95%;
```

```
 max-width: 500px;
```

```
 background: white;
```

```
 border-radius: 12px;
```

```
 box-shadow: 0 10px 25px rgba(0,0,0,0.3);
```

```
 display: flex;
```

```
 flex-direction: column;
```

```
 overflow: hidden;
```

```
}
```

```
.chat-header {
 background: #4CAF50;
 color: white;
 padding: 15px;
 text-align: center;
 font-weight: bold;
}
```

```
.chat-box {
 flex: 1;
 padding: 15px;
 overflow-y: auto;
 background: #f4f4f4;
}
```

```
.message {
 margin-bottom: 10px;
 padding: 10px;
 border-radius: 8px;
 max-width: 80%;
}
```

```
.user {
 background: #DCF8C6;
 align-self: flex-end;
}
```

```
.bot {
 background: #e2e2ff;
 align-self: flex-start;
}
```

```
.input-area {
 display: flex;
 border-top: 1px solid #ddd;
}
```

```
input {
 flex: 1;
 padding: 10px;
 border: none;
 outline: none;
}
```

```
button {
 padding: 10px 15px;
 border: none;
 background: #4CAF50;
 color: white;
 cursor: pointer;
}
```

```
button:hover {
 background: #45a049;
}
</style>
</head>

<body>

<div class="chat-container">
 <div class="chat-header">FireDrop Inspired AI Website Assistant</div>

 <div id="chatBox" class="chat-box"></div>

 <div class="input-area">
 <input type="text" id="userInput" placeholder="Type your message...">
 <button onclick="sendMessage()">Send</button>
 </div>
</div>

<script>
const chatBox = document.getElementById("chatBox");

// Add message to UI
function addMessage(text, sender) {
 const message = document.createElement("div");
 message.classList.add("message", sender);
```

```
message.innerText = text;

chatBox.appendChild(message);
chatBox.scrollTop = chatBox.scrollHeight;
}

// Send message
function sendMessage() {
 const input = document.getElementById("userInput");
 const userText = input.value.trim().toLowerCase();

 if (!userText) return;

 addMessage(input.value, "user");

 setTimeout(() => {
 const botReply = getBotResponse(userText);
 addMessage(botReply, "bot");
 }, 600);

 input.value = "";
}

// Bot logic
function getBotResponse(input) {
```

```
if (input.includes("contact")) {
 return "Suggestion: Add a Contact section with Name, Email, and Message
form.";
}
```

```
if (input.includes("blog")) {
 return "Add a Blog section to improve SEO and share updates.";
}
```

```
if (input.includes("generate content")) {
 return "Sample Content: We are a passionate team delivering quality
solutions to grow your business online.";
}
```

```
if (input.includes("home page")) {
 return "Your homepage should include a Hero section, CTA button, and
overview.";
}
```

```
if (input.includes("seo")) {
 return "Use proper headings, meta tags, alt text, and fast-loading design for
SEO.";
}
```

```
if (input.includes("help")) {
 return "I can help you design sections, generate content, and improve
structure.";
}
```

```
 return "Try asking about sections like Contact, About, Services, Blog,
Portfolio or SEO.";
```

```
}
```

```
// Initial message
```

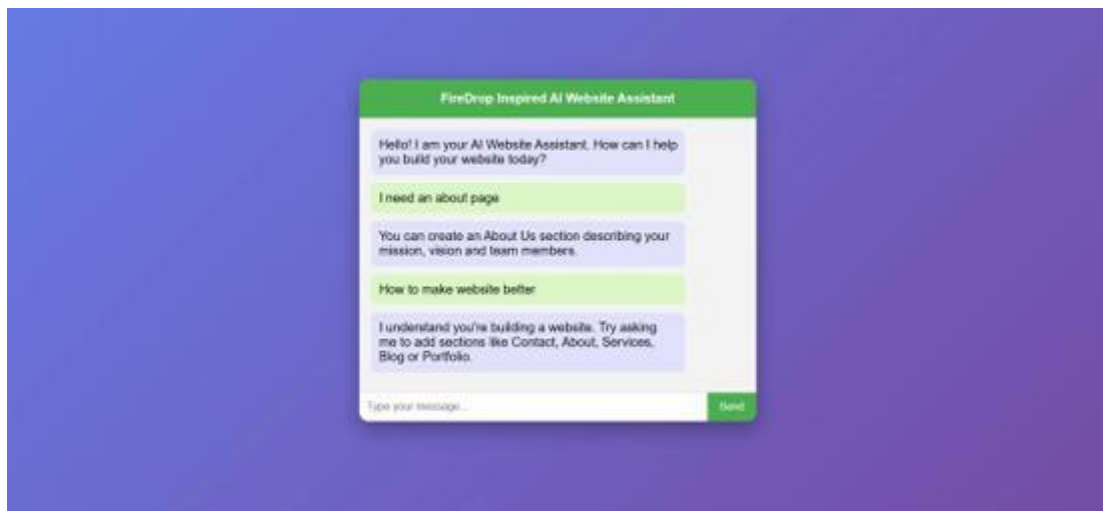
```
addMessage("Hello! I am your AI Website Assistant. How can I help you build
your website today?", "bot");
```

```
</script>
```

```
</body>
```

```
</html>
```

## OUTPUT



## RESULT

Thus, the AI chatbot webpage was implemented successfully and the output was verified.